

---

# Sabaragamuwa University of Sri Lanka



## Assignment

|                     |                                                |
|---------------------|------------------------------------------------|
| <b>Title</b>        | University Student Information Exchange System |
| <b>Course Title</b> | Semantic Web                                   |
| <b>Course Code</b>  | DS5103                                         |
| <b>Student Name</b> | Mohomed Yoosuf                                 |
| <b>Index Number</b> | 22CDS0403                                      |

DEPARTMENT OF DATA SCIENCE  
FACULTY OF COMPUTING

11<sup>th</sup> September 2026

---

# Table of Contents

|                                                                       |   |
|-----------------------------------------------------------------------|---|
| Table of Contents .....                                               | 1 |
| List of Images .....                                                  | 2 |
| List of Tables .....                                                  | 2 |
| Attributes vs. Elements .....                                         | 3 |
| XML Tree for One Student .....                                        | 4 |
| Well Formed vs. Valid .....                                           | 5 |
| Three XML errors that could cause a System to reject a document ..... | 5 |
| How XML Helps Exchange Data Between Heterogeneous Systems .....       | 6 |

## List of Images

|                                          |   |
|------------------------------------------|---|
| Image 1 : XML Tree for One Student ..... | 4 |
|------------------------------------------|---|

## List of Tables

|                                                                                 |   |
|---------------------------------------------------------------------------------|---|
| Table 1 : Three XML errors that could cause a System to reject a Document ..... | 5 |
|---------------------------------------------------------------------------------|---|

## Attributes vs. Elements

The rule I followed was simple: if a value is primary content that someone would actually want to read — a name, an email, a course title — it became an element. If a value is really just a compact identifier, classification tag, or bit of metadata used to describe or look up a record, it became an attribute. That's why:

- `id` on `<student>` is an attribute — it's a lookup key, not something you'd “read” as content, and XSD lets it be pattern-validated (e.g. `S001`) independently of the element's children.
- `faculty`, `department`, `year` and `semester` on `<academicInfo>` are attributes — they classify the record rather than being the record's substance, and there's exactly one of each per student, which is the classic attribute use case.
- `code` and `credits` on `<course>` are attributes for the same reason — short, single-valued identifiers used to reference a specific course.
- `fullName`, `email`, `dateOfBirth`, `title`, `lecturer` and `grade` all stayed as elements, because they're the actual data being exchanged — they can be more complex, are more likely to be queried or extended later, and (unlike attributes) elements can be nested further if a system needs to break them down more, e.g. splitting `fullName` into `firstName/lastName`.

# XML Tree for One Student

The tree below shows student S001 (Kamal Perera). student is the root of this subtree with one attribute (@id) and three child elements — personalInfo, academicInfo and courses. Each of those branches out further: personalInfo into three leaf elements, academicInfo into its four attributes plus a programme child, and courses into repeated course elements, each of which has its own attributes and three leaf children (title, lecturer, grade).

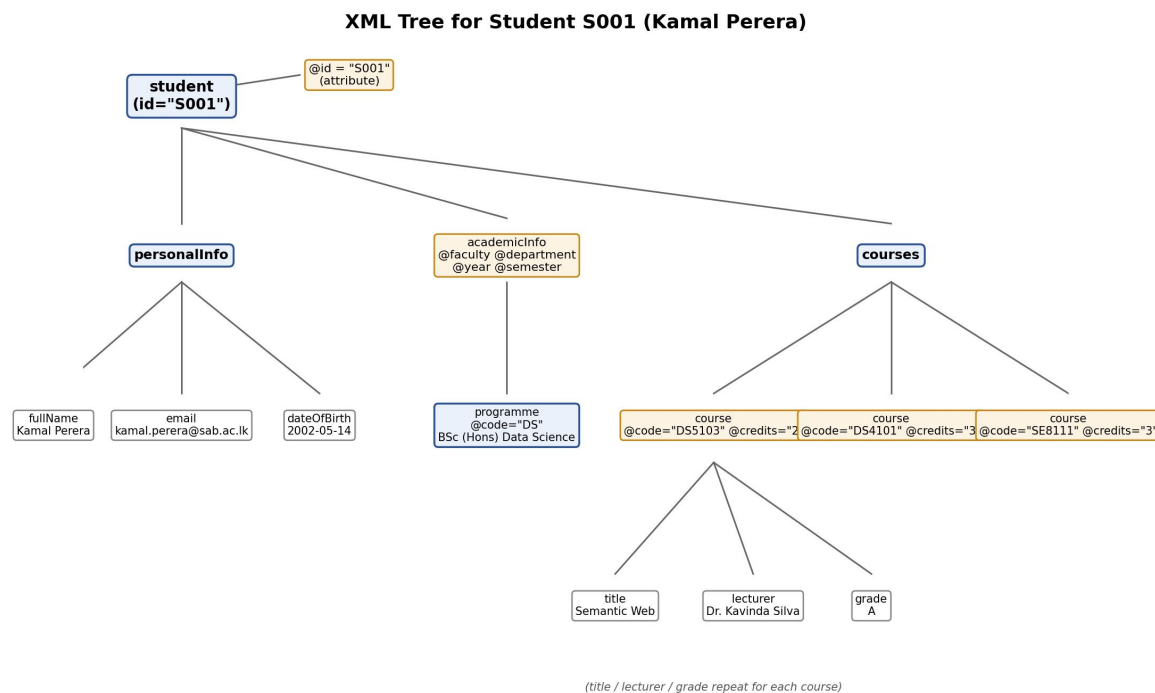


Image 1: XML Tree for One Student

## Well Formed vs. Valid

My `student_data.xml` being well-formed just means it plays by the basic rules of XML syntax: there's exactly one root element (`studentRecords`), every tag that opens is closed, tags are properly nested rather than overlapping, attribute values are quoted, and names are used consistently with matching case throughout. A parser can read a well-formed document without throwing a syntax error — but that says nothing about whether the content actually makes sense. Valid is a stronger claim: it means the document is well-formed and it conforms to `student_data.xsd`. That schema adds rules well-formedness alone can't check — that every student must have an `id`, that `year` has to be between 1 and 4, that `dateOfBirth` has to actually be a date, and that `grade` can only be one of a fixed set of letter grades. I validated my XML against the XSD (using `lxml`) and it passes; if I deleted a required `<email>` element or set `year="9"`, the file would remain well-formed but would fail validation, which is exactly the gap DTD/XSD checking exists to catch before the data reaches another system.

## Three XML errors that could cause a System to reject a document

| # | Error                                                                                                  | Why it would be rejected                                                                                                                                                                                                                           |
|---|--------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 1 | Mismatched tag case, e.g. <code>&lt;uni:Student&gt;</code> closed by <code>&lt;/uni:student&gt;</code> | XML is case-sensitive, so the parser sees two different element names that never close — the document fails to parse at all (not even well-formed).                                                                                                |
| 2 | Missing the required <code>id</code> attribute on a <code>&lt;student&gt;</code> element               | The document could still be well-formed, but <code>student_data.xsd</code> marks <code>id</code> as <code>use="required"</code> , so schema validation fails and an interoperable system relying on the XSD would reject it.                       |
| 3 | A grade value outside the allowed set, e.g. <code>&lt;uni:grade&gt;A+&lt;/uni:grade&gt;</code>         | “A+” is well-formed text, but it isn't one of the enumerated values in <code>GradeType</code> , so it's syntactically fine yet semantically invalid — the same class of problem as the <code>credits="ABC"</code> example from the lecture slides. |

Table 1: Three XML errors that could cause a System to reject a Document

# How XML Helps Exchange Data Between Heterogeneous Systems

The Student Management, Learning Management, Examination and Faculty systems almost certainly store data differently internally — different databases, different field names, maybe different platforms entirely. XML gives them a common, self-describing format to meet in the middle: a tag like `<email>` or `<grade>` names what the data means, not just how one particular system happens to store it, so any system that receives the file can interpret it correctly without needing to know the sender's internal schema. Because the structure is explicit and hierarchical, related data — a student's courses, grades and programme — travels together instead of being scattered across separate exports. Adding a namespace (uni:) further prevents collisions if another system's XML also happens to use a tag like `<course>` for something else entirely. And because the format can be validated against a shared XSD, all four systems can agree in advance on exactly what a “valid” student record looks like, catching malformed or incomplete data before it ever gets imported — which is really the whole point of exchanging structured data instead of ad hoc text files.